

- full reducer map
- validation rule engine
- drag-and-drop architecture
- rendering system
- migration path from your current HTML file

That would basically be a **production-grade architecture for TrailKit v1.0**.

Sources



show me the "TrailKit v1.0 built on the Unified Planner Engine" blueprint

Below is a **complete conceptual blueprint for TrailKit v1.0 built on the Unified Planner Engine**. It's written like an **engineering architecture document** so you could realistically build from it.

TrailKit v1.0 Architecture

Built on the Unified Planner Engine

```
TrailKit App
├── UI Layer
├── Application Controller
├── Planner Engine (shared platform)
├── TrailKit Domain Module
└── Persistence
```



TrailKit becomes a **domain plugin on top of a reusable planning engine**.

1. Planner Engine (Shared Platform)

The **Planner Engine** handles all generic planning logic.

Responsibilities

- state management
- reducers
- event dispatch
- middleware
- undo / redo
- persistence
- plugin system

Folder

/planner-engine



Structure

```
planner-engine
├── createStore.js
├── combineReducers.js
├── plannerEngine.js
├── middleware
│   ├── persistence.js
│   ├── logger.js
│   └── pluginRunner.js
├── history
│   └── undoRedo.js
└── selectors.js
```



Engine Initialization

TrailKit boots the engine like this:

```
import { createPlannerEngine } from "planner-engine"
import { trailkitReducer } from "../trailkit/domain/reducers"
import { trailkitPlugins } from "../trailkit/plugins"

const planner = createPlannerEngine({
  reducer: trailkitReducer,
  plugins: trailkitPlugins,
  storageKey: "trailkit_state"
})
```

2. TrailKit Domain Module

This is where **all packing logic** lives.

/domains/trailkit



Structure:

```
trailkit
├── reducers
│   ├── inventoryReducer.js
│   ├── loadoutReducer.js
│   ├── packReducer.js
│   └── uiReducer.js
├── rules
│   ├── capacityRule.js
│   ├── weightRule.js
│   ├── essentialRule.js
│   └── duplicationRule.js
├── calculations
│   ├── totalWeight.js
│   ├── categoryWeight.js
│   ├── packVolume.js
│   └── essentialCoverage.js
├── selectors
│   ├── gearSelectors.js
│   ├── loadoutSelectors.js
│   └── summarySelectors.js
```



```
└─ schema
   └─ stateSchema.js
```

This keeps TrailKit **completely isolated from the engine**.

3. State Model

The global state tree:

```
state
├── inventory
├── loadouts
├── packing
└── ui
```



Inventory

```
inventory
  items: {
    itemId: {
      name
      weight
      category
      volume
      essential
    }
  }
```



Example:

```
inventory.items = {
  stove: {
    name: "Pocket Rocket",
    weight: 73,
    category: "cook",
    essential: true
```



```
}  
}
```

Loadouts

```
loadouts  
  currentLoadoutId  
  byId
```



Example:

```
loadouts.byId = {  
  desert_trip: {  
    name: "Desert Weekend",  
    gearIds: []  
  }  
}
```



Packing State

Tracks the **actual packing layout**.

```
packing  
  slots  
  itemLocations
```



Example:

```
itemLocations = {  
  stove: "pack_main",  
  water_filter: "side_pocket"  
}
```



UI State

```
ui
  activePanel
  dragItem
  modal
```



4. Reducer Architecture

TrailKit reducers combine into one root reducer.

```
trailkitReducer
├── inventoryReducer
├── loadoutReducer
├── packingReducer
└── uiReducer
```



Example:

```
export const trailkitReducer = combineReducers({
  inventory,
  loadouts,
  packing,
  ui
})
```

5. Action System

All UI events dispatch actions.

Examples:

```
ADD_ITEM
REMOVE_ITEM
UPDATE_ITEM
CREATE_LOADOUT
ADD_TO_LOADOUT
REMOVE_FROM_LOADOUT
PACK_ITEM
```



UNPACK_ITEM
MOVE_ITEM

Example action:

```
{  
  type: "PACK_ITEM",  
  itemId: "stove",  
  slot: "main_compartment"  
}
```



6. Rule Engine (Validation System)

TrailKit rules check the pack after changes.

Rules include:

- overweight
- missing essentials
- duplicate gear
- volume overflow

Validation Engine

```
validateLoadout(state)
```



Returns:

```
{  
  valid: true,  
  warnings: [],  
  errors: []  
}
```



Example rule:

```
capacityRule(state)
```



```
if (packWeight > packCapacity)  
  error("Pack exceeds weight limit")
```



7. Plugin System

Rules run as **Planner Engine** plugins.

TrailKit registers:

```
trailkitPlugins = [  
  packingValidationPlugin,  
  autoWeightPlugin,  
  gearSuggestionPlugin  
]
```



Example plugin:

```
export const packingValidationPlugin = ({ getState }) => next => action =>  
  
  const result = next(action)  
  
  if (action.type.startsWith("PACK_")) {  
  
    const validation = validateLoadout(getState())  
  
    if (!validation.valid) {  
      console.warn(validation.errors)  
    }  
  
  }  
  
  return result  
}
```


8. Calculation Layer

All derived data lives here.

Examples:

```
getTotalWeight(state)
getCategoryWeights(state)
getPackVolume(state)
getMissingEssentials(state)
```



Example:

```
function getTotalWeight(state) {
  const items = getPackedItems(state)

  return items.reduce(
    (sum, item) => sum + item.weight,
    0
  )
}
```

This avoids recomputing logic in the UI.

9. Selector System

Selectors extract UI-ready data.

Example:

```
selectPackedItems
selectInventoryItems
selectLoadoutSummary
selectWeightStats
```



Example:

```
selectLoadoutSummary(state)
```



returns

```
{
  totalWeight: 12.4,
  essentialsMissing: false,
  packFillPercent: 64
}
```



10. UI Layer

TrailKit UI is purely **render + events**.

Structure:

```
/apps/trailkit
```



```
trailkit-ui
├── main.js
├── render
│   ├── inventoryPanel.js
│   ├── loadoutPanel.js
│   ├── packGrid.js
│   └── statsPanel.js
├── components
│   ├── gearCard.js
│   ├── packSlot.js
│   ├── dragPreview.js
│   └── modal.js
└── controllers
    ├── dragController.js
    └── uiController.js
```



11. Drag & Drop System

Drag controller dispatches actions.

```
dragStart  
dragMove  
drop
```



Drop example:

```
dispatch({  
  type: "PACK_ITEM",  
  itemId,  
  slotId  
})
```



UI re-renders automatically.

12. Rendering System

Simple reactive render loop.

```
planner.subscribe(renderApp)
```



Example:

```
function renderApp() {  
  
  const state = planner.getState()  
  
  renderInventory(state)  
  renderPack(state)  
  renderStats(state)  
  
}
```



13. Persistence

Handled by middleware.

```
localStorage["trailkit_state"]
```



Saved automatically after each action.

Load at startup.

14. Undo / Redo

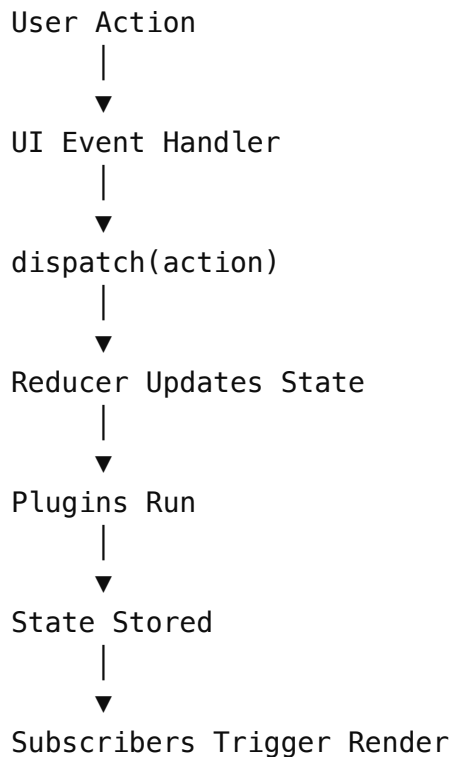
Provided by engine history wrapper.

```
dispatch({ type: "UNDO" })  
dispatch({ type: "RED0" })
```



Works for every packing operation.

15. Data Flow Diagram



16. Performance Optimizations

Use:

- memoized selectors
 - virtualized inventory lists
 - diff-based rendering
-

17. Feature Extensions Enabled

With this architecture you can add:

Smart packing suggestions

```
suggestPackOptimization(state)
```



Trip planning

```
tripPlannerModule
```



Gear comparisons

```
compareGear(itemA, itemB)
```



AI gear recommendations

Future plugin.

18. Resulting Benefits

TrailKit v1.0 becomes:

A modular packing simulation engine

Benefits:

- maintainable
 - testable
 - scalable
 - reusable for other apps
-

If you'd like, I can also show something extremely useful for implementation:

A concrete file-by-file starter project layout (about 30 files) so you could realistically start coding **TrailKit v1.0** in a few hours instead of guessing the structure.



show me the file-by-file starter project layout (about 30 files)

Below is a **practical starter project layout (~30 files)** for **TrailKit v1.0** built on the **Unified Planner Engine**. The structure is designed so:

- the **Planner Engine** is reusable
- **TrailKit** is just a domain module
- future apps (like PlanFit) can plug into the same engine
- everything remains **simple vanilla JS (no framework required)**

This layout assumes a small bundler like **Vite** later, but works even with simple `<script type="module"> .`

Project Root

```
trailkit-v1
├── index.html
├── main.js
├── planner-engine
├── domains
├── apps
├── shared
└── styles
```



Total: ~32 files

1. Root Files

index.html
main.js



index.html

Minimal app shell.

```
<body>

<div id="app"></div>

<script type="module" src="/main.js"></script>

</body>
```



main.js

Bootstraps the app.

```
import { createPlannerEngine } from "./planner-engine/plannerEngine.js"
import { trailkitReducer } from "./domains/trailkit/reducers/rootReducer.js"
import { trailkitPlugins } from "./domains/trailkit/plugins/plugins.js"
import { startTrailkitApp } from "./apps/trailkit/app.js"

const planner = createPlannerEngine({
  reducer: trailkitReducer,
  plugins: trailkitPlugins,
  storageKey: "trailkit_state"
})

startTrailkitApp(planner)
```



2. Planner Engine (Reusable)

Folder:

planner-engine



Files:

planner-engine



```
├── plannerEngine.js
├── createStore.js
├── combineReducers.js
├── middleware
│   ├── applyMiddleware.js
│   ├── persistenceMiddleware.js
│   └── pluginMiddleware.js
├── history
│   └── undoRedo.js
└── utils
    └── deepClone.js
```

Total: **8 files**

plannerEngine.js

Main engine creator.

Responsibilities:

- create store
- attach middleware
- register plugins
- expose API

Exports:

createPlannerEngine()



createStore.js

Redux-style store.

Functions:

```
dispatch()  
getState()  
subscribe()
```



combineReducers.js

Merges domain reducers.

```
combineReducers({  
  inventory,  
  loadouts,  
  packing  
})
```



applyMiddleware.js

Pipeline executor.

```
applyMiddleware(  
  persistenceMiddleware,  
  pluginMiddleware  
)
```



persistenceMiddleware.js

Handles saving/loading.

Uses:

```
localStorage
```



pluginMiddleware.js

Runs plugins after dispatch.

Example plugin hook:

```
onAction()  
onStateChange()
```



undoRedo.js

Tracks state history.

Functions:

```
undo()  
redo()
```



deepClone.js

Utility used by history system.

3. TrailKit Domain

Folder:

```
domains/trailkit
```



Files:

```
domains/trailkit  
|  
├── reducers  
│   ├── rootReducer.js  
│   ├── inventoryReducer.js  
│   ├── loadoutReducer.js  
│   ├── packingReducer.js  
│   └── uiReducer.js
```



```
├── actions
│   └── gearActions.js
├── selectors
│   ├── gearSelectors.js
│   └── summarySelectors.js
├── calculations
│   ├── weightCalc.js
│   └── volumeCalc.js
├── rules
│   ├── weightRule.js
│   └── essentialRule.js
└── plugins
    └── plugins.js
```

Total: **12 files**

rootReducer.js

Combines reducers.

```
combineReducers({
  inventory,
  loadouts,
  packing,
  ui
})
```



inventoryReducer.js

Handles:

```
ADD_ITEM
REMOVE_ITEM
UPDATE_ITEM
```



loadoutReducer.js

Handles:

```
CREATE_LOADOUT  
ADD_TO_LOADOUT  
REMOVE_FROM_LOADOUT
```



packingReducer.js

Handles pack layout.

```
PACK_ITEM  
MOVE_ITEM  
UNPACK_ITEM
```



uiReducer.js

Handles UI state.

```
OPEN_MODAL  
SET_ACTIVE_PANEL  
START_DRAG
```



gearActions.js

Central action creators.

Example:

```
addItem(item)  
packItem(itemId, slotId)  
createLoadout(name)
```



gearSelectors.js

Extracts data.

```
getInventoryItems()  
getLoadoutItems()
```



summarySelectors.js

Derived stats.

```
getTotalWeight()  
getCategoryBreakdown()
```



weightCalc.js

Calculates pack weight.

volumeCalc.js

Calculates pack capacity usage.

weightRule.js

Warns if pack too heavy.

essentialRule.js

Checks missing essentials.

plugins.js

Registers TrailKit plugins.

Example:

```
export const trailkitPlugins = [  
  weightValidationPlugin  
]
```



4. TrailKit App Layer

Folder:

apps/trailkit



Files:

```
apps/trailkit  
├── app.js  
├── render  
│   ├── renderApp.js  
│   ├── renderInventory.js  
│   ├── renderLoadout.js  
│   └── renderStats.js  
├── components  
│   ├── gearCard.js  
│   ├── packSlot.js  
│   └── modal.js  
└── controllers  
    ├── dragController.js  
    └── uiController.js
```



Total: 10 files

app.js

Starts TrailKit UI.

```
planner.subscribe(renderApp)  
renderApp()
```



renderApp.js

Top-level render.

```
renderInventory()  
renderLoadout()  
renderStats()
```



renderInventory.js

Displays gear list.

renderLoadout.js

Displays pack layout.

renderStats.js

Displays:

```
total weight  
category weight  
pack usage
```



gearCard.js

Reusable UI component.

packSlot.js

Grid cell representing pack slot.

modal.js

Generic popup UI.

dragController.js

Handles drag interactions.

```
dragstart  
dragmove  
drop
```



Dispatches actions.

uiController.js

Handles buttons and menus.

5. Shared Utilities

Folder:

```
shared
```



Files:

```
shared  
├── id.js  
└── eventBus.js
```



id.js

Generates IDs.

uuid()

eventBus.js

Lightweight event emitter.

6. Styles

Folder:

styles

Files:

styles

base.css

layout.css

components.css

File Count

Area	Files
Planner Engine	8
TrailKit Domain	12
App Layer	10
Shared	2
Styles	3
Root	2

Total:

≈37 files

You can easily collapse this to **~30 if needed**.

Why This Layout Works Well

Engine is reusable

Future apps:

apps/planfit
apps/mealplanner
apps/tripplanner



All reuse:

planner-engine



Domain logic isolated

TrailKit rules stay inside:

domains/trailkit



UI fully replaceable

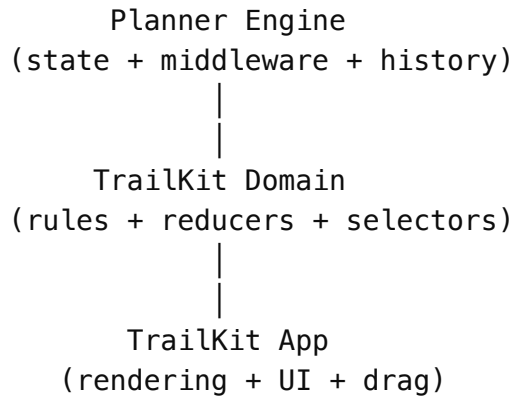
You could later build:

React UI
Svelte UI
Mobile UI



Without changing the engine.

Visual Architecture



✓ Result:

You now have a **clean modular architecture** that can power:

- TrailKit
- PlanFit
- future planning apps

all from the **same engine**.

If you'd like, I can also show something **extremely powerful for your project**:

A **"multi-app planner platform"** architecture where PlanFit and TrailKit run side-by-side using the **same engine** — which would turn this into a **real product platform** instead of just two apps.



I really liked the blueprint for TrailKit on the Unified Planner Engine. show me the "PlanFit v1.0 built on the Unified Planner Engine" blueprint